MNIST Digit Recognition

Three models compared: SVM, MLP and CNN

1. Overview

Three models were trained to recognise handwritten digits (0-9) from the MNIST dataset: a classical machine-learning model (a Support Vector Machine, **SVM**), a neural network (a multi-layer perceptron, **MLP**) and a deep-learning model (a convolutional neural network, **CNN**). They were trained and tested on the same data, compared, and the best one was chosen. All three are available in a small web application where the user draws a digit and gets a prediction.

2. Data and preprocessing

- **Data.** MNIST has 70,000 grey-scale images of handwritten digits, 28 x 28 pixels each. The dataset is split into 60,000 training images and 10,000 test images. The notebooks download it automatically.
- **Pixel scaling.** Every pixel (0-255) is divided by 255, so values are between 0 and 1. The digit is white on a black background, as in MNIST. No data augmentation and no other normalisation were used.
- **Train / validation / test split.** The 60,000 training images are split into **54,000 for training** and **6,000 for validation**. The split is stratified (each digit keeps its share) and uses a fixed random seed (42). The validation set is used to compare settings and to choose the best training epoch. The **10,000 test images are used only once**, for the final evaluation.
- **Input shape.** The SVM and the MLP receive each image flattened into 784 numbers. The CNN receives it as a 1 x 28 x 28 array (one grey channel), so it can use the two-dimensional shape of the digit.
- **Drawings in the app.** A drawn digit is prepared like MNIST digits: it is cropped to the digit, scaled to fit a 20 x 20 box, and centred in a 28 x 28 image by its centre of mass. Then the pixels are scaled to 0-1.

3. Hyperparameter choices

Each model was tuned with a small search. Every setting was trained on the training split and scored on the validation split; the test set was never used. The table shows what was searched, what was chosen and why.

Model	Searched	Chosen	Why
SVM (RBF kernel)	C = 1, 5, 10 and gamma = 0.01, 0.02, 0.04 (9 runs). Each run used a random sample of 10,000 training images, because SVM training is slow.	**C = 5, gamma = 0.02** (validation accuracy 97.23% on the sample)	Best of the nine. gamma = 0.02 was best for both C = 5 and C = 10, and C = 10 gave no gain (97.22%), so the best value is inside the grid and not at its edge. The RBF kernel is the standard choice for pixel images because it gives curved boundaries between digits.
MLP	Hidden layers (128), (256, 128), (512, 256) x learning rate 0.01, 0.001 (6 runs, 5 epochs each). Fixed: Adam optimiser, batch size 128, dropout 0.3.	**Layers (512, 256), learning rate 0.001** (validation accuracy 97.82%)	With learning rate 0.001, wider layers were better (96.70%, 97.62%, 97.82%). Learning rate 0.01 was too large for the two bigger networks (96.47%, 96.10%). Dropout 0.3 was not tuned: it is a cheap guard against overfitting, which MLPs are prone to.
CNN	Filters (8, 16), (16, 32), (32, 64) x learning rate 0.001, 0.003 (6 runs, 3 epochs each). Fixed: 3 x 3 filters, 2 x 2 max-pooling, 64 dense units, Adam, batch size 128, no dropout.	**Filters (32, 64), learning rate 0.003** (validation accuracy 98.57%)	More filters were better at both learning rates, and 0.003 beat 0.001 for every size (partly because three epochs are short, so a faster learner has an advantage). No dropout was used: the CNN has few parameters and the smallest train-test gap.

Final training. The SVM was retrained on all 54,000 training images with the chosen settings. The MLP (up to 20 epochs) and the CNN (up to 10 epochs) were retrained on the same images; after every epoch the validation accuracy is checked and the weights of the best epoch are kept (early stopping).

4. Training results and comparison

All numbers are for the final models on the 10,000 test images (train accuracy: 10,000 of the training images). Training time is for the final training run on a CPU, without a GPU; the hyperparameter searches are not included. The best values are in bold (train accuracy is shown for reference).

Model	Test accuracy	Test errors (of 10,000)	Train accuracy	Train-test gap (points)	Training time	Parameters	File size
SVM	98.51%	149	99.99%	1.48	95 s	9.76 million*	5.8 MB
MLP	98.36%	164	99.77%	1.41	**9.5 s**	535,818	2.1 MB
CNN	**99.02%**	**98**	99.55%	**0.53**	85 s	**220,234**	**0.9 MB**

* For the SVM, "parameters" is the number of values it stores: 12,453 training images (support vectors) of 784 pixels each.

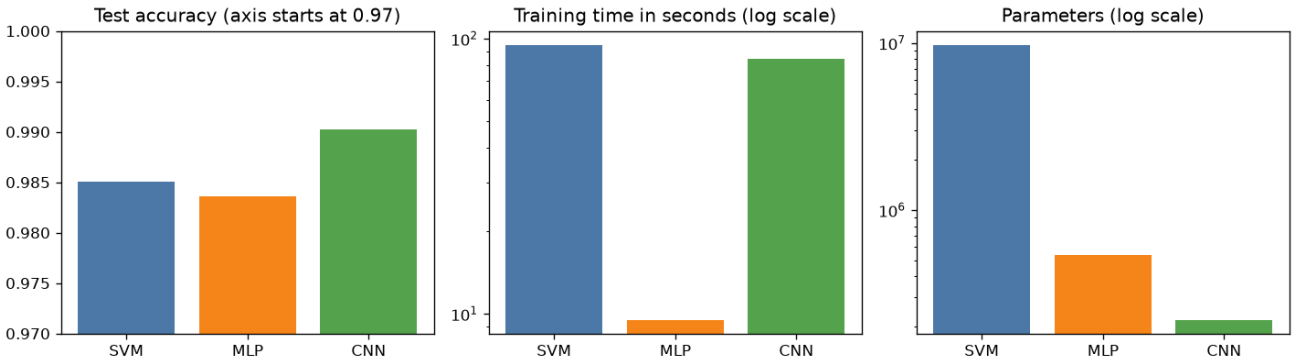

Figure 1. Test accuracy (the axis starts at 0.97), training time and number of parameters for the three models.

5. Choosing the best model: the CNN

The CNN is the best choice for digit recognition.

- **Performance.** It has the highest test accuracy, 99.02%: 98 mistakes in 10,000 images, against 149 for the SVM and 164 for the MLP.
- **Generalisation.** The gap between train and test accuracy is the smallest, 0.53 points, against 1.48 (SVM) and 1.41 (MLP). The SVM almost memorises the training images (99.99%) but does worse on new ones.
- **Complexity.** The CNN has 220,234 parameters, fewer than the MLP (535,818). The SVM stores 12,453 training images, so every prediction compares the new image with all of them. The CNN's file is also the smallest (0.9 MB).
- **Training time.** The MLP is the fastest (9.5 s). The CNN needs 85 s, about the same as the SVM (95 s). Training happens only once, so this is an acceptable cost for the better accuracy.
- **Suitability for the task.** Recognising a digit means recognising shapes in an image. The CNN's filters look for strokes and curves wherever they appear, so it uses the 2-D structure that the SVM and the MLP throw away when they flatten the image. This matters for hand-drawn digits, which vary in position, thickness and slant.

Limits of this comparison. Each model was trained once (seed 42), and the searches were small (6-9 runs per model), so the difference between the SVM and the MLP (15 test images) should not be over-interpreted. The CNN's lead is larger: 51 fewer mistakes than the SVM and 66 fewer than the MLP.

6. Files and how to run

- **Training code:** notebooks/01_svm.ipynb, 02_mlp.ipynb and 03_cnn.ipynb. 04_comparison.ipynb builds the comparison table and chart.
- **Saved models:** app/models/classical_svm.joblib, mlp_model.pt and cnn_model.pt.
- **Application:** the app/ folder (app.py, inference.py, model_utils.py, image_utils.py, templates/ and static/). The run instructions are in README.md; in short: **python app/app.py**, then open http://127.0.0.1:5000.